BRECQ: PUSHING THE LIMIT OF POST-TRAINING QUANTIZATION BY BLOCK ECONSTRUCTION

26/03/26

BRECQ (PTQ)

[기존 문제]

1. Parameter 공간의 근사는 task 성능을 보장하지 않는다.

Quantization Background Uniform symmetric quantization maps the floating-point numbers to several fixed-points. These points (or grids) have the same interval and are symmetrically distributed. We denote the set that contains these grids as $\mathcal{Q}_b^{\text{u,sym}} = s \times \{-2^{b-1}, \ldots, 0, \ldots, 2^{b-1}-1\}$. Here, s is the step size between two grids and b is the bit-width. Quantization function, denoted by $q(\cdot) : \mathcal{R} \to \mathcal{Q}_b^{\text{u,sym}}$, is generally designed to minimize the quantization error:

$$\min ||\hat{\mathbf{w}} - \mathbf{w}||_F^2, \text{ s.t. } \hat{\mathbf{w}} \in \mathcal{Q}_b^{\text{u,sym}} \tag{2}$$

Solving this minimization problem, one can easily get the $q(\cdot)$ by leveraging the rounding-to-nearest operation $\lfloor \cdot \rceil$. Rounding-to-nearest is a prevalent method to perform quantization, e.g. PACT (Choi et al., 2018). However, recently some empirical or theoretical evidence supports that simply minimizing the quantization error in parameter space does not bring optimal task performances. Specif-

⇒ Task Loss에 기반한 quantization error를 도입하자.

2. 직접 task loss를 최적화하는 것은 QAT는 가능하지만, PTQ로는 달성하기 어렵다.

⇒ 따라서, 현실적인 cost로 인해 task loss 자체를 최소화하지는 못하지만, loss의 증가를 최소화하는 방식으로 PTQ Quantization을 진행하자.

is minimized without re-training the weights. Their motivations are all towards minimizing a final objective, which is the task loss, i.e.,

$$\min \mathbb{E}[L(\hat{\mathbf{w}})], \text{ s.t. } \hat{\mathbf{w}} \in \mathcal{Q}_b^{\text{u,sym}}. \tag{3}$$

While this optimization objective is simple and can be well-optimized in QAT scenarios, it is not easy to learn the quantized weight without end-to-end finetuning as well as sufficient training data and computing resources. In post-training quantization settings, we only have full precision weights that $\mathbf{w}^\star = \arg\min_{\mathbf{w}} \mathbb{E}[L(\mathbf{w})]$ where $\mathbf{w} \in \mathcal{R}$ and a small subset of training data to do calibration.

Taylor Expansion It turns out that the quantization imposed on weights could be viewed as a special case of weight perturbation. To quantitatively analyze the loss degradation caused by quantization, Nagel et al. (2020) use Taylor series expansions and approximates the loss degradation by

$$\mathbb{E}[L(\mathbf{w} + \Delta\mathbf{w})] - \mathbb{E}[L(\mathbf{w})] \approx \Delta\mathbf{w}^\mathsf{T} \bar{\mathbf{g}}^{(\mathbf{w})} + \frac{1}{2} \Delta\mathbf{w}^\mathsf{T} \bar{\mathbf{H}}^{(\mathbf{w})} \Delta\mathbf{w}, \tag{4}$$

충분히 학습된 모델의 경우 g = 0에 근사되므로, 해당 문제는 AdaRound에서도 언급했던 wHw를 최소화하는 문제로 귀결된다.

3. AdaRound에서 Hessian을 **단순 [c x I]**로 상수 취급하는 방식으로 인해, **cross-layer 간 dependency를 고려하지 않게 되고**, 해당 방식으로 인한 오차가 BRECQ에서 해결하고자 하는 main problem인 **"Second-Order problem"**이다.

1. Layers are mutual-independent, thus the Hessian is in the form of layer-diagonal[1] and Kronecker-factored, i.e., $\bar{\mathbf{H}}^{(\mathbf{w}^{(\ell)})} = \mathbb{E}[\mathbf{x}^{(\ell)}\mathbf{x}^{(\ell),\mathsf{T}} \otimes \mathbf{H}^{(\mathbf{z}^{(\ell)})}]$, where $\otimes$ is the Kronecker product.
2. The second-order derivatives of pre-activations are constant diagonal matrix ($\mathbf{H}^{(\mathbf{z}^{(\ell)})} = c \times \mathbf{I}$) which is independent of input data points.

At last, the objective is transformed into a practical proxy signal, *the change in feature-maps* ($\mathbf{z} = \mathbf{W}\mathbf{x}$), and the quantized model can be obtained by a layer-by-layer feature map reconstruction algorithm (with few calibration images). Recent works, like Bit-Split (Wang et al., 2020) and AdaQuant (Hubara et al., 2020), also take this layer-wise objective to improve the post-training quantization. However, they failed to quantize weights to INT2. We think the inherent reason is that when $\Delta\mathbf{w}$ grows higher, the former assumptions do not hold and an accurate signal is required.

Delata_w가 커질 수록, layer 간의 quantization error를 독립적으로 다루는 것에 대한 drawback이 점차 커진다. 이로 인해, 8→4→2 bit 수를 줄여 나갈 수록, 이에 따른 delta_w가 자연스레 커지기 때문에, 기존 방식들에서는 저비트 양자화에 실패한 것을 알 수 있다.

<AdaRound 부가설명>

Quantization 된 weights를 weight perturbation처럼 생각할 수 있고 Nagel et al. (2020)에서는 Taylor series expansions를 이용해 quantization에 의한 loss degradation을 다음과 같이 근사한다.

$$E[L(\mathbf{w}+\Delta\mathbf{w})] - E[L(\mathbf{w})] \approx \Delta\mathbf{w}^T\bar{\mathbf{g}}(\mathbf{w}) + \tfrac{1}{2}\Delta\mathbf{w}^T\bar{\mathbf{H}}(\mathbf{w})\Delta\mathbf{w}$$

여기서 $\bar{\mathbf{g}}^{(\mathbf{w})} = E[\nabla_{\mathbf{w}}L]$이고 $\bar{\mathbf{H}}^{(\mathbf{w})} = E[\nabla^2_{\mathbf{w}}L]$이다. 또 $\Delta\mathbf{w}$는 weight perturbation이다.

pre-trained model이 minimum으로 converge하기 때문에 FONC에 의해 $\bar{\mathbf{g}}^{(\mathbf{w})}$는 $\mathbf{0}$이라 생각할 수 있고 아래와 같이 second-order approximation형태로 나타낼 수 있다.

$$E[L(\mathbf{w}+\Delta\mathbf{w})] - E[L(\mathbf{w})] \approx \tfrac{1}{2}\Delta\mathbf{w}^T\bar{\mathbf{H}}(\mathbf{w})\Delta\mathbf{w}$$

여기서 한 가지 문제점이 발생하는데, full Hessian은 많은 device에서 memory infeasiable 하다는 것이다. (terabytes of memory space를 필요로 한다.)

Nagel et al. (2020)에서는 이를 아래의 두 가지 가정을 이용해 해결 한다.

1. Layers가 mutal-independet 하다는 것이다. 따라서 Hessian을 layer-diagonal 형태로 표현하고 $\bar{\mathbf{H}}^{(\mathbf{w}^{(l)})} \approx E[\mathbf{x}^{(l)}\mathbf{x}^{(l),T} \otimes \mathbf{H}^{(\mathbf{z}^{(l)})}]$과 같이 근사한다. 여기서 $\otimes$ 는 Kronecker product이다.

2. pre-activations의 Hessian은 Input data와 무관하게 constant diagonal matrix로 근사한다. ($\mathbf{H}^{(\mathbf{z}^{(l)})} = c \times \mathbf{I}$)

다양한 study에서 이러한 layer-wise objective를 사용하였으나 모두 INT2까지 weights를 quantize 하는데 실패하였다. 저자는 이를 $\Delta\mathbf{w}$가 커질수록, 위의 가정이 성립하지 않기 때문이라고 봤고 더 정확한 근사가 필요하다고 보았다.

*이에 본 논문은, **layer 단위 reconstruction에서 block 단위의 reconstruction**을 제시하며, "layer간의 의존성"을 반영하는 동시에 오버피팅 되지 않도록 "generalization"을 확보한다.*

[Method]

1. **Cross-Layer Dependency**

기존 AdaRound에서는 Hessian을 단순 C x I 를 통해 diagonalize하여 연산을 단순화하였다.

하지만, 이로 인해 서로 다른 layer의 의존성을 무시했다.

BRECQ에서는 Hessian의 계산 복잡성을 해결하기 위해, 모델 가중치에 대한 2차 미분 계수를 **출력의 변화에 따른 2차 미분계수로** 재정의한다.

유도 과정은 다음과 같다.

LHS: Hessian (모델 가중치의 변화에 대한 2차 미분계수)

Denote the neural network output $\mathbf{z}^{(n)} = f(\theta)$, the loss function can be represented by $L(f(\theta))$ where $\theta = \mathrm{vec}[\mathbf{w}^{(1),\mathsf{T}}, \ldots, \mathbf{w}^{(n),\mathsf{T}}]^{\mathsf{T}}$ is the stacked vector of weights in all n layers. The Hessian matrix can be computed by

$$\frac{\partial^2 L}{\partial\theta_i \partial\theta_j} = \frac{\partial}{\partial\theta_j}\left(\sum_{k=1}^{m} \frac{\partial L}{\partial \mathbf{z}_k^{(n)}} \frac{\partial \mathbf{z}_k^{(n)}}{\partial\theta_i}\right) = \sum_{k=1}^{m} \frac{\partial L}{\partial \mathbf{z}_k^{(n)}} \frac{\partial^2 \mathbf{z}_k^{(n)}}{\partial\theta_i \partial\theta_j} + \sum_{k,l=1}^{m} \frac{\partial \mathbf{z}_k^{(n)}}{\partial\theta_i} \frac{\partial^2 L}{\partial \mathbf{z}_k^{(n)} \partial \mathbf{z}_l^{(n)}} \frac{\partial \mathbf{z}_l^{(n)}}{\partial\theta_j}, \quad (5)$$

where $\mathbf{z}^{(n)} \in \mathcal{R}^m$. Since the pretrained full precision model is converged to a local minimum, we can assume the Hessian is positive-semidefinite (PSD). Specifically, the converged model has $\nabla_{\mathbf{z}^{(n)}} L$ close to $\mathbf{0}$ so the first term in Eq. (5) is neglected and Hessian becomes the Gauss-Newton (GN) matrix $\mathbf{G}^{(\theta)}$. GN matrix can be written in matrix form (Botev et al., 2017) as

$$\mathbf{H}^{(\theta)} \approx \mathbf{G}^{(\theta)} = \mathbf{J}_{\mathbf{z}^{(n)}}(\theta)^{\mathsf{T}} \mathbf{H}^{(\mathbf{z}^{(n)})} \mathbf{J}_{\mathbf{z}^{(n)}}(\theta), \quad (6)$$

Chain Rule에 따라 좌항을 우항과 같이 분해할 수 있고 (eq.5), 우항의 왼쪽 항은 앞서 말한 바와 같이, 충분히 학습된 모델의 경우 0으로 근사할 수 있다.

따라서, 최종적인 Hessain은 eq.6과 같이 정리 가능하다.

이를 이전의 wHw0에 대입하면, (이 논문에서, w는 delta_theta로 표현하면, 다음과 같이 식이 유도된다.

Substituting above equation in Eq. (12), we have

$$\Delta\theta^{\top}\mathbf{H}^{(\theta)}\Delta\theta = \sum_{i=1}^{d}\sum_{j=1}^{d}\Delta\theta_i\Delta\theta_j\left(\sum_{k=1}^{m}\sum_{l=1}^{m}\frac{\partial\mathbf{z}_k^{(n)}}{\partial\theta_i}\frac{\partial^2 L}{\partial\mathbf{z}_k^{(n)}\mathbf{z}_l^{(n)}}\frac{\partial\mathbf{z}_l^{(n)}}{\partial\theta_j}\right) \tag{14a}$$

$$= \sum_{i=1}^{d}\sum_{j=1}^{d}\sum_{k=1}^{m}\sum_{l=1}^{m}\left(\Delta\theta_i\Delta\theta_j\frac{\partial\mathbf{z}_k^{(n)}}{\partial\theta_i}\frac{\partial^2 L}{\partial\mathbf{z}_k^{(n)}\mathbf{z}_l^{(n)}}\frac{\partial\mathbf{z}_l^{(n)}}{\partial\theta_j}\right) \tag{14b}$$

$$= \sum_{k=1}^{m}\sum_{l=1}^{m}\left(\frac{\partial^2 L}{\partial\mathbf{z}_k^{(n)}\mathbf{z}_l^{(n)}}\right)\left(\sum_{i=1}^{d}\Delta\theta_i\frac{\partial\mathbf{z}_k^{(n)}}{\partial\theta_i}\right)\left(\sum_{j=1}^{d}\Delta\theta_j\frac{\partial\mathbf{z}_k^{(n)}}{\partial\theta_j}\right) \tag{14c}$$

$$= (\Delta\theta\mathbf{J}\left[\frac{\mathbf{z}^{(n)}}{\theta}\right])^{\top}\mathbf{H}^{(\mathbf{z}^{(n)})}(\Delta\theta\mathbf{J}\left[\frac{\mathbf{z}^{(n)}}{\theta}\right]), \tag{14d}$$

where we define the $\mathbf{J}[\frac{x}{y}]$ is the Jacobian matrix of x w.r.t. y. To this end, we use the first-order Taylor expansion as we did in Eq. (8) to approximate the change in network output, i.e.,

$$\Delta\mathbf{z}^{(n)} \approx \Delta\theta\mathbf{J}\left[\frac{\mathbf{z}^{(n)}}{\theta}\right] \tag{15}$$

Therefore, the final objective is transformed to $\Delta\mathbf{z}^{(n),\top}\mathbf{H}^{(\mathbf{z}^{(n)})}\Delta\mathbf{z}^{(n)}$. □

즉, wHw 가 최종적으로 zHz로 변환할 수 있다.

(여기서 z는 출력값을 의미한다.)

⇨ Network의 final output z의 reconstruction을 통해 quantization으로 인한 discrepency를 최소화 할 수 있다. 즉, quantization의 목적을 network distillation으로 볼 수 있다. Full precision teacher model을 low precision student model에 distillation 하는…

- 왜 wHw 보다 zHz가 나은가?
  - 모델 내 가중치는 수 만개에서, 수 억개 이상이 된다. wHw의 항은 모든 weight, 즉 가중치에 대한 변화율을 계산해야 되기 때문에, (10000 x 10000)이 된다. [이것도 작게 잡은 것…]
  - 따라서, cost가 매우 크다.
  - 하지만, z로 변환할 경우, 해당 모델을 통과한 출력값의 개수는 가중치의 개수에 비해 현저히 적다. 아래 그림만 봐도, 특정 layer안의 가중치는 1000개라고 하더라도 해당 layer의 output은 1개이다. 벌써 계산양을 1000배 줄일 수 있다.

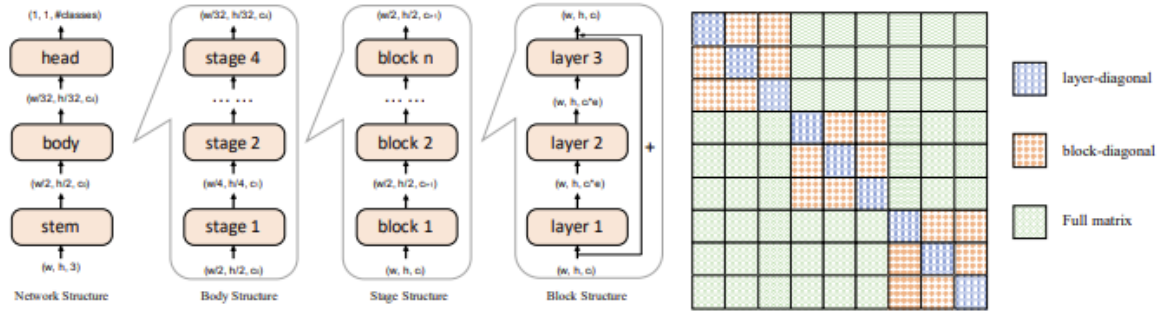
2. Block Reconstruction

하지만, Full: network output reconstruction으로 진행하게 될 경우, 오히려 성능이 떨어진다.

이는 calibration set에 overfitting되어 발생하는 것으로, parameter의 수가 dataset의 수보다 많아 calibration dataset에 과적합 된 것이다.

오히려, layer-wise reconstruction에서 각 layer를 독립적으로 reconstruction하는 것이 regularization처럼 작용하여, generalization error를 감소시킨다.

이에 layer와 full 각각이 regularization과 dependency에 대한 trade off가 존재하여, 이의 sweet spot인 block wise reconstruction을 택한다.



(a) A typical structure of CNN (taken from Radosavovic et al. (2020)). Network is composed of a stem layer (first convolution on input images), a body and a head layer (average pooling with a fully connected layer). A body contains several stages, and a stage contains several blocks. A representative block is the bottleneck block with residual path.

(b) An example illustration of Hessian (or Fisher) matrix. Blue sub-block means the layer-diagonal and each layer are mutual-independent; orange sub-block consider the dependency inside a building block and green parts measure all dependencies.

Figure 1: We define 4 kinds of reconstruction granularity, namely net-wise, stage-wise, block-wise and layer-wise optimization, each of them corresponds an essential component of CNN.

The layer-wise optimization corresponds to layer-diagonal Hessian (Fig. 1b blue parts) and the network-wise optimization corresponds to full Hessian (Fig. 1b green parts). Similarly, we can define an intermediate block-diagonal Hessian. Formally, if layer k to layer ℓ (where $1 \leq k < \ell \leq n$) form a block, the weight vector is defined as $\tilde{\theta} = \mathrm{vec}[\mathbf{w}^{(k),\mathsf{T}}, \ldots, \mathbf{w}^{(\ell),\mathsf{T}}]^{\mathsf{T}}$ and the Hessian can be also transformed by $\Delta\tilde{\theta}^{\mathsf{T}}\tilde{\mathbf{H}}^{(\tilde{\theta})}\Delta\tilde{\theta} = \mathbb{E}[\Delta\mathbf{z}^{(\ell),\mathsf{T}}\mathbf{H}^{(\mathbf{z}^{(\ell)})}\Delta\mathbf{z}^{(\ell)}]$. Such block-diagonal Hessian ignores the inter-block dependency and considers the intra-block dependency but it produces less generalization error. Then we can block-by-block reconstruct the intermediate output.

본문에서 알 수 있 듯, kth~lth layer를 하나의 block으로 잡으면,

최종적으로 lth layer를 통과한 output에 대한 변화율을 바탕으로 optimization을 진행한다.

이러한 block-diagonal hessian은 inter-block(다른 block) 간의 의존성은 무시하되, intra-block (본인 block) 내의 의존성은 고려하여, regularization과 dependency의 trade-off를 고려한다.

3. Approximating Pre-Activation Hessian

Block wise reconstruction에 대한 zHz는 다음과 같이 계산한다.

Block 내부에서 기존 AdaRound의 Hessian을 CxI로 근사하는 것은 각각의 가중치를 동일한 중요도로 처리한다.

BRECQ에서는 Fisher Information Matrix를 통해 개별적인 중요도를 반영하도록, 단순 C x I의 대각행렬이 아닌, 각각의 민감도를 반영한 대각행렬을 도입한다.

이게 가능한 것은, 모델이 충분히 수렴했다면, Hessian == FIM 즉, 1차 미분의 제곱값이기 때문에 단순하게 유도할 수 있다.

$$\bar{\mathbf{F}}^{(\theta)} = \mathbb{E}\left[\nabla_\theta \log p_\theta(y|x) \nabla_\theta \log p_\theta(y|x)^\mathsf{T}\right] = -\mathbb{E}\left[\nabla_\theta^2 \log p_\theta(y|x)\right] = -\bar{\mathbf{H}}^{(\theta)}_{\log p(x|\theta)}. \quad (9)$$

The FIM is equal to the negative expected Hessian of the log-likelihood function, therefore, a simple corollary is that the Hessian of task loss will become FIM if the model distribution matches the true data distribution (LeCun et al., 2012). Although matching true data distribution seems impossible, this is the best we can do since the pretrained model is converged.

The diagonal of the pre-activation FIM is equal to the squared gradients of each elements, which is successfully used in Adam (Kingma & Ba, 2014) for the second momentum. The optimization objective becomes

$$\min_{\hat{\mathbf{w}}} \mathbb{E}\left[\Delta\mathbf{z}^{(\ell),\mathsf{T}} \mathbf{H}^{(\mathbf{z}^{(\ell)})} \Delta\mathbf{z}^{(\ell)}\right] = \min_{\hat{\mathbf{w}}} \mathbb{E}\left[\Delta\mathbf{z}^{(\ell),\mathsf{T}} \mathrm{diag}\left((\frac{\partial L}{\partial \mathbf{z}_1^{(\ell)}})^2, \ldots, (\frac{\partial L}{\partial \mathbf{z}_a^{(\ell)}})^2\right) \Delta\mathbf{z}^{(\ell)}\right]. \quad (10)$$

BRECQ는 어떠한 optimization method에 적용 가능하며,

weight에는 AdaRound를, activation에는 LSQ를 사용했다.

Algorithm 1: BRECQ optimization

Input: Pretrained FP model; Calibration dataset, iteration T
for *all* $i = 1, 2, \ldots, N$*-th block in the FP model* **do**
  Collect input data to the block $\mathbf{x}^{(i)}$, the FP output $\mathbf{z}^{(i)}$ and its gradient $\mathbf{g}^{(\mathbf{z}^{(i)})}$;
  for *all* $j = 1, 2, \ldots, T$*-iteration* **do**
    Get quantized output $\hat{\mathbf{z}}^{(i)}$ and compute $\Delta\mathbf{z}^{(i)} = \mathbf{z}^{(i)} - \hat{\mathbf{z}}^{(i)}$;
    Descend Eq. (10) and update the rounding of all the weights in this block (Eq. (16));
    if *Activation Quantization is triggered* **then**
      Update the activation quantization step size (Eq. (18)).
  After optimization, compute the sensitivity for each layer and between layers (2-bit only);
return Quantized model, Sensitivities for mixed precision;

이 과정에서 2, 4, 8 bit에 대해 block wise로 개별적은 Loss Optimization을 수행하고, 이후 Mixed Precision에 사용하기 위한 LookUp table을 만든다.

4. Mixed Precision

주어진 룩업 테이블을 바탕으로, 유전 알고리즘 기반의 Mixed Precision을 수행한다.

$$\min_{\mathbf{c}} L(\hat{\mathbf{w}}, \mathbf{c}), \quad \text{s.t. } H(\mathbf{c}) \leq \delta, \quad \mathbf{c} \in \{2, 4, 8\}^n.$$

H 즉, 지연시간, 모델 크기 제약을 만족하면서, Loss를 최소화하는 precision 조합을 구상한다.

미리 만들어둔 룩업 테이블을 가지고 유전 알고리즘만 수행하면 되기 때문에 소요시간이 매우 적으며, Loss는 전체 block의 Loss를 sum한 값을 활용한다.

이로 인해 기존의 3^n에 대한 시간을 3초로 단축한다.

[Experiments]

Table 1: Ablation study.

Model	Layer	Block	Stage	Net
ResNet-18	65.19	**66.39**	66.01	54.15
MobileNetV2	52.13	**59.67**	54.23	40.76

2-bit quantization에 대한 결과이며, Block-wise reconstruction이 가장 좋은 성능을 나타내는 것을 확인할 수 있다.

Table 2: Accuracy comparison on *weight-only quantized* post-training models. Activations here are unquantized and kept full precision. We also conduct variance study for our experiments. Bold values indicates best results. * indicates our implementation based on open-source codes.

Methods	Bits (W/A)	ResNet-18	ResNet-50	MobileNetV2	RegNet-600MF	RegNet-3.2GF	MnasNet-2.0
Full Prec.	32/32	71.08	77.00	72.49	73.71	78.36	76.68
Bias Correction*	4/32	50.43	64.64	62.82	67.09	71.73	72.31
OMSE (Choukroun et al., 2019)	4/32	67.12	74.67	-	-	-	-
AdaRound (Nagel et al., 2020)	4/32	68.71	75.23	69.78	71.97*	77.12*	74.87*
AdaQuant (Hubara et al., 2020)	4/32	68.82	75.22	44.78	-	-	-
Bit-Split (Wang et al., 2020)	4/32	69.11	75.58	-	-	-	-
BRECQ (Ours)	4/32	**70.70**±0.07	**76.29**±0.04	**71.66**±0.04	**73.02**±0.09	**78.04**±0.04	**76.00**±0.02
Bias Correction*	3/32	12.85	7.97	10.89	28.82	17.95	40.72
AdaRound (Nagel et al., 2020)*	3/32	68.07	73.42	64.33	67.71	72.31	69.33
AdaQuant (Hubara et al., 2020)*	3/32	58.12	67.61	12.56	-	-	-
Bit-Split (Wang et al., 2020)	3/32	66.75	73.24	-	-	-	
BRECQ (Ours)	3/32	**69.81**±0.05	**75.61**±0.09	**69.50**±0.12	**71.48**±0.07	**77.22**±0.04	**74.58**±0.08
Bias Correction*	2/32	0.13	0.12	0.14	0.18	0.11	0.11
AdaRound (Nagel et al., 2020)*	2/32	55.96	47.95	32.54	25.66	24.70	30.60
AdaQuant (Hubara et al., 2020)*	2/32	0.30	0.49	0.11	-	-	-
BRECQ (Ours)	2/32	**66.30**±0.12	**72.40**±0.12	**59.67**±0.13	**65.83**±0.13	**73.88**±0.14	**67.13**±0.13

Table 3: Accuracy comparison on *fully quantized* post-training models. Activations here are quantized to 4-bit. Notations follows the upper table.

Methods	Bits (W/A)	ResNet-18	ResNet-50	MobileNetV2	RegNet-600MF	RegNet-3.2GF	MNasNet-2.0
Full Prec.	32/32	71.08	77.00	72.49	73.71	78.36	76.68
ACIQ-Mix (Banner et al., 2019)	4/4	67.0	73.8	-	-	-	-
ZeroQ (Cai et al., 2020)*	4/4	21.71	2.94	26.24	28.54	12.24	3.89
LAPQ (Nahshan et al., 2019)	4/4	60.3	70.0	49.7	57.71*	55.89*	65.32*
AdaQuant (Hubara et al., 2020)	4/4	67.5	73.7	34.95*	-	-	-
Bit-Split (Wang et al., 2020)	4/4	67.56	73.71	-	-	-	
BRECQ (Ours)	4/4	**69.60**±0.04	**75.05**±0.09	**66.57**±0.67	**68.33**±0.28	**74.21**±0.19	**73.56**±0.24
ZeroQ (Cai et al., 2020)*	2/4	0.08	0.08	0.10	0.10	0.05	0.12
LAPQ (Nahshan et al., 2019)*	2/4	0.18	0.14	0.13	0.17	0.12	0.18
AdaQuant (Hubara et al., 2020)*	2/4	0.21	0.12	0.10	-	-	
BRECQ (Ours)	2/4	**64.80**±0.08	**70.29**±0.23	**53.34**±0.15	**59.31**±0.49	**67.15**±0.11	**63.01**±0.35

다양한 조합에서 기존의 성능들을 모두 상회하는 것을 확인할 수 있다.

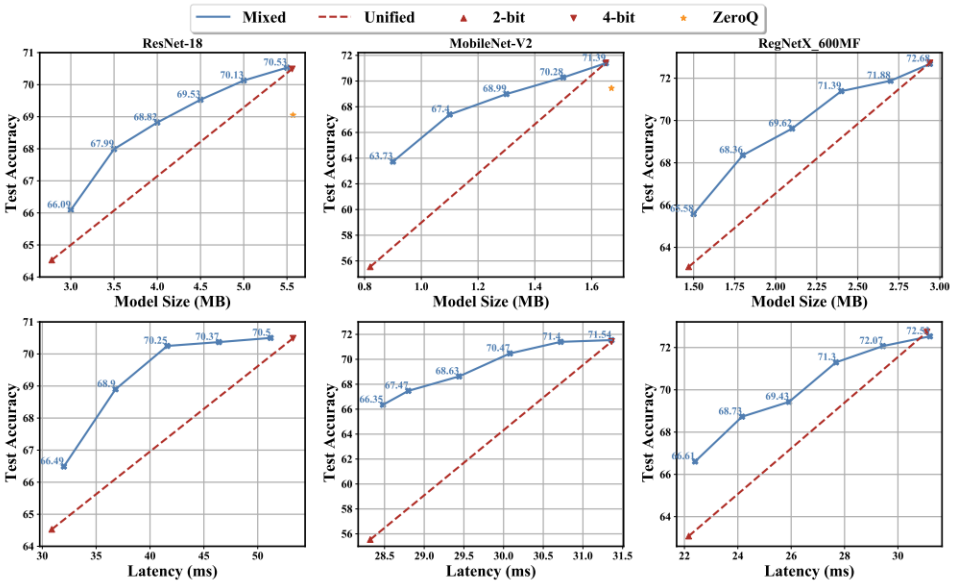


Figure 2: Mixed Precision results.

Mixed Precision > Unified Precision

<Mixed Precision 추가설명>

BRECQ에서 가중치(Weight)와 활성화 값(Activation)에 서로 다른 양자화 방식을 사용하는 이유는 **두 데이터의 성질이 근본적으로 다르기 때문**입니다 [1]. 결론부터 말씀드리면, **두 곳 모두에 AdaRound를 똑같이 적용하는 것은 불가능**합니다 [1].

구체적인 이유는 다음과 같습니다.

1. 가중치(Weight) - AdaRound 사용 가능

가중치는 이미 학습이 끝난 모델에 **고정되어 있는 정적인 값**입니다 [2].

- 값이 변하지 않기 때문에, 각 가중치 요소마다 개별적으로 "이 값은 올림(Ceil)을 할지, 내림(Floor)을 할지" 학습 가능한 변수(v)를 통해 결정하고 이를 영구적으로 고정해 둘 수 있습니다 [1] [2].

2. 활성화(Activation) - AdaRound 사용 불가, LSQ 도입

반면, 활성화 값은 네트워크에 어떤 입력 데이터(이미지 등)가 들어오느냐에 따라 **매번 실시간으로 계속 변하는 동적인 값**입니다 [1].

- 들어오는 값 자체가 시시각각 바뀌기 때문에, 특정 위치의 값에 대해 "무조건 올림해라" 혹은 "무조건 내림해라"라는 식의 고정된 반올림 정책을 적용할 수 없습니다 [1].
- 따라서 개별 값의 반올림 방향을 학습하는 대신, **양자화 격자의 간격(Step size, s)을 넓히거나 좁혀서 전체적인 오차를 줄이는 LSQ 방식**을 채택한 것입니다 [1].

요약하자면: 가중치는 고정된 값이라 '각각의 값을 어떻게 반올림할지' 정해둘 수 있지만, 활성화 값은 계속 변하기 때문에 반올림 방향을 고정할 수 없어 '전체 눈금의 간격'을 조절하는 다른 최적화 전략(LSQ)을 사용해야만 합니다 [1].